FLEETLOGIX · PROYECTO INTEGRADOR DE DATA SCIENCE

Manual de Consultas SQL

12 queries de negocio con explicación técnica
y análisis de optimización mediante índices

BASE DE DATOS	PostgreSQL — fleetlogix
VOLUMEN DE DATOS	~505.000 registros (100k trips, 400k deliveries)
QUERIES DOCUMENTADAS	12 (8 de negocio + 4 avanzadas con window functions)
ÍNDICES EVALUADOS	5 índices creados sobre trips, deliveries, maintenance, drivers y routes
MÉTODO	EXPLAIN ANALYZE antes y después de crear los índices

Vero Iacono · Data Science
Julio 2026

Índice

1. Introducción y metodología
2. Índices de optimización creados
3. Las 12 queries: detalle técnico y de negocio
3.1 Query 1 — Conteo de vehículos por tipo
3.2 Query 2 — Conductores con licencia próxima a vencer (60 días)
3.3 Query 3 — Total de viajes por estado
3.4 Query 4 — Entregas por ciudad destino (últimos 2 meses)
3.5 Query 5 — Conductores activos con viajes completados
3.6 Query 6 — Promedio de entregas por conductor (últimos 6 meses)
3.7 Query 7 — Rutas con mayor consumo de combustible por km
3.8 Query 8 — Entregas retrasadas por día de la semana
3.9 Query 9 — Costo de mantenimiento por kilómetro recorrido
3.10 Query 10 — Ranking de conductores por eficiencia (Window Functions)
3.11 Query 11 — Tendencia de viajes con LAG y LEAD
3.12 Query 12 — Pivot de entregas por hora y día de la semana
4. Resumen comparativo de performance
5. Conclusiones y recomendaciones

1. Introducción y metodología

Este manual documenta las 12 consultas SQL desarrolladas para FleetLogix, el sistema de gestión de flota y logística sobre el cual se construyó el pipeline ETL del proyecto. Cada query resuelve un problema de negocio concreto (cumplimiento legal, monitoreo operativo, planificación de demanda, productividad de conductores, eficiencia de combustible, costos y proyección de tendencias) sobre una base de datos PostgreSQL con seis tablas relacionadas: vehicles, drivers, routes, trips, deliveries y maintenance.

La base contiene aproximadamente 505.000 registros distribuidos de forma no uniforme entre las tablas: 200 vehículos, 400 conductores, 48 rutas, 100.000 viajes, 400.000 entregas y 5.000 mantenimientos. Esta asimetría es relevante para interpretar los resultados de performance: las queries que tocan trips o deliveries trabajan sobre órdenes de magnitud mucho mayores que las que solo consultan vehicles o drivers.

Metodología de evaluación

Cada una de las 12 queries se ejecutó dos veces con EXPLAIN ANALYZE: una primera vez contra la base sin índices adicionales (más allá de los tres índices básicos del esquema original), y una segunda vez después de crear los cinco índices de optimización descritos en la sección 2 y correr ANALYZE sobre las seis tablas. En ambos casos se registraron el plan de ejecución completo y el Execution Time reportado por Postgres, permitiendo comparar tanto el tiempo total como la estrategia de acceso elegida por el optimizador (Seq Scan vs. Index Scan, tipo de join, uso de memoria vs. disco en los sorts, etc.).

2. Índices de optimización creados

Con base en el análisis de los planes de ejecución de las 12 queries, se diseñaron cinco índices orientados a los patrones de acceso más frecuentes: filtros por estado, joins entre las tablas grandes (trips, deliveries) y filtros temporales.

Índice	Definición	Objetivo	Queries beneficia
idx_trips_composite_joins	`trips(vehicle_id, driver_id, route_id, departure_datetime) WHERE status = 'completed'`	Acelerar los JOINs entre trips y vehicles/drivers/routes, filtrando solo viajes completados.	4, 5, 6, 7, 9, 10, 11
idx_deliveries_scheduled_datetime	`deliveries(scheduled_datetime, delivery_status) WHERE delivery_status = 'delivered'`	Optimizar filtros temporales sobre entregas ya completadas.	4, 8, 12
idx_maintenance_vehicle_cost	`maintenance(vehicle_id, cost)`	Acceso rápido a costos de mantenimiento agrupados por vehículo.	9
idx_drivers_status_license	`drivers(status, license_expiry) WHERE status = 'active'`	Filtrar conductores activos y sus vencimientos de licencia sin escanear inactivos.	2, 5, 6, 10
idx_routes_metrics	`routes(route_id, distance_km, destination_city)`	Cubrir joins y cálculos de distancia/ciudad sin acceder al heap de routes.	4, 7, 9, 10

Tras crear los índices se ejecutó ANALYZE sobre las seis tablas para actualizar las estadísticas del planificador y asegurar que las decisiones de Seq Scan vs. Index Scan reflejen el estado real de los datos.

3. Las 12 queries: detalle técnico y de negocio

Query 1 — Conteo de vehículos por tipo

Problema de negocio: *Conocer la composición de la flota para decisiones de renovación y compra.*

```
EXPLAIN ANALYZE
SELECT v.vehicle_type, COUNT(v.vehicle_type)
FROM vehicles v
GROUP BY v.vehicle_type
ORDER BY COUNT(v.vehicle_type) DESC;
```

Plan de ejecución (antes de índices)

HashAggregate sobre Seq Scan de vehicles (200 filas). El motor agrupa por vehicle_type en una sola pasada en memoria (Batches: 1, 32kB) y ordena el resultado (4 filas) con quicksort.

Efecto de los índices de optimización

vehicles es una tabla pequeña (200 filas) que cabe en un puñado de páginas, así que Postgres descarta cualquier índice y usa Seq Scan por ser más barato — comportamiento esperado y correcto del optimizador. Ninguno de los 5 índices creados aplica a esta tabla.

Antes de índices	Después de índices	Variación
0,181 ms	0,169 ms	▼ 6.6%

Veredicto: SIN CAMBIO (esperado)

Query 2 — Conductores con licencia próxima a vencer (60 días)

Problema de negocio: *Prevenir problemas legales por licencias vencidas.*

```
EXPLAIN ANALYZE
SELECT
    driver_id, employee_code,
    first_name || ' ' || last_name AS nombre_completo,
    license_number, license_expiry,
    (license_expiry - CURRENT_DATE) AS dias_para_vencer,
    phone, status
FROM drivers
WHERE status = 'active'
  AND license_expiry BETWEEN CURRENT_DATE AND CURRENT_DATE + INTERVAL '60 days'
ORDER BY license_expiry ASC;
```

Plan de ejecución (antes de índices)

Seq Scan sobre drivers (400 filas) con filtro combinado de status y license_expiry, descartando 381 filas y devolviendo 19. El resultado se ordena con quicksort en memoria (26kB).

Efecto de los índices de optimización

El índice idx_drivers_status_license (status, license_expiry) WHERE status='active' está pensado exactamente para este filtro. Aun así, con solo 400 filas totales y 19 resultados, el planner sigue prefiriendo Seq Scan porque leer la tabla completa es más barato que la suma de acceso al índice + lookup a la tabla (heap fetch) para tan pocas filas.

Antes de índices	Después de índices	Variación
0,267 ms	0,239 ms	▼ 10.5%

Veredicto: SIN CAMBIO (esperado)

Query 3 — Total de viajes por estado

Problema de negocio: *Monitorear operaciones en curso.*

```
EXPLAIN ANALYZE
SELECT
    status,
    COUNT(*) AS total_viajes,
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS porcentaje
FROM trips
GROUP BY status
ORDER BY total_viajes DESC;
```

Plan de ejecución (antes de índices)

HashAggregate sobre los 100.000 registros de trips, seguido de una WindowAgg para calcular el porcentaje sobre el total con SUM(COUNT(*)) OVER (). Todo el trabajo se resuelve en memoria (17kB de storage para la ventana).

Efecto de los índices de optimización

Esta query no filtra por columna alguna: agrupa el 100% de la tabla trips, así que cualquier índice es irrelevante — siempre va a requerir leer todas las filas (Seq Scan). El leve cambio en el tiempo de ejecución (41.7ms → 39.5ms) es ruido normal entre corridas, no efecto de los índices.

Antes de índices	Después de índices	Variación
41,725 ms	39,512 ms	▼ 5.3%

Veredicto: SIN CAMBIO (esperado)

Query 4 — Entregas por ciudad destino (últimos 2 meses)

Problema de negocio: *Identificar demanda por ciudad para planificación de recursos.*

```
EXPLAIN ANALYZE
SELECT
    r.destination_city AS ciudad_destino,
    COUNT(*) AS total_entregas,
    COUNT(*) FILTER (WHERE d.delivery_status = 'delivered') AS entregadas,
    COUNT(*) FILTER (WHERE d.delivery_status != 'delivered') AS pendientes_o_fallidas,
    ROUND(100.0 * COUNT(*) FILTER (WHERE d.delivery_status = 'delivered') / COUNT(*), 2) AS
pct_cumplimiento
FROM deliveries d
JOIN trips t ON d.trip_id = t.trip_id
JOIN routes r ON t.route_id = r.route_id
WHERE d.scheduled_datetime >= CURRENT_DATE - INTERVAL '2 months'
GROUP BY r.destination_city
ORDER BY total_entregas DESC;
```

Plan de ejecución (antes de índices)

Antes de los índices: Parallel Seq Scan sobre deliveries (400.000 filas, descartando 193.077 por el filtro de fecha) combinado con Hash Join contra routes y trips. El plan usa 2 workers paralelos y tarda ~159ms.

Efecto de los índices de optimización

idx_deliveries_scheduled_datetime (scheduled_datetime, delivery_status) WHERE delivery_status='delivered' reduce el volumen de filas a inspeccionar para el filtro de fecha reciente, y idx_trips_composite_joins agiliza el join contra trips. Resultado: 158.8ms → 136.4ms, una mejora real de ~14%, la más significativa del set.

Antes de índices	Después de índices	Variación
158,797 ms	136,430 ms	▼ 14.1%

Veredicto: MEJORA

Query 5 — Conductores activos con viajes completados

Problema de negocio: *Evaluar carga de trabajo por conductor.*

```
EXPLAIN ANALYZE
SELECT
    dr.driver_id, dr.employee_code,
    dr.first_name || ' ' || dr.last_name AS conductor,
    COUNT(t.trip_id) FILTER (WHERE t.status = 'completed') AS viajes_completados
FROM drivers dr
LEFT JOIN trips t ON dr.driver_id = t.driver_id
WHERE dr.status = 'active'
GROUP BY dr.driver_id, dr.employee_code, dr.first_name, dr.last_name
ORDER BY viajes_completados ASC;
```

Plan de ejecución (antes de índices)

Hash Right Join entre trips (Seq Scan, 100.000 filas) y drivers (Seq Scan filtrado a 373 activos). Postgres arma la tabla hash con drivers (más chica) y recorre trips una sola vez.

Efecto de los índices de optimización

El plan sigue usando Seq Scan en ambas tablas: drivers es demasiado chica para beneficiarse de idx_drivers_status_license, y trips no tiene filtro por status en el WHERE (el FILTER está en el SELECT, no reduce el Seq Scan). El tiempo pasó de 82.5ms a 87.9ms, una variación de ~6% dentro del rango de ruido esperado para un Seq Scan completo de 100k filas.

Antes de índices	Después de índices	Variación
82,509 ms	87,894 ms	▲ 6.5%

Veredicto: SIN MEJORA

Query 6 — Promedio de entregas por conductor (últimos 6 meses)

Problema de negocio: *Medir productividad individual de conductores.*

```
EXPLAIN ANALYZE
SELECT
    dr.driver_id, dr.employee_code,
    dr.first_name || ' ' || dr.last_name AS conductor,
    COUNT(DISTINCT t.trip_id) AS viajes_realizados,
    COUNT(del.delivery_id) AS entregas_totales,
    ROUND(COUNT(del.delivery_id)::NUMERIC / NULLIF(COUNT(DISTINCT t.trip_id), 0), 2) AS
entregas_promedio_por_viaje
FROM drivers dr
JOIN trips t ON dr.driver_id = t.driver_id
JOIN deliveries del ON del.trip_id = t.trip_id
WHERE dr.status = 'active'
  AND t.departure_datetime >= CURRENT_DATE - INTERVAL '6 months'
GROUP BY dr.driver_id, dr.employee_code, dr.first_name, dr.last_name
ORDER BY entregas_promedio_por_viaje DESC;
```

Plan de ejecución (antes de índices)

Con el índice, el plan usa Index Scan idx_trips_departure sobre trips (20.637 filas via índice de fecha) en vez de recorrer las 100.000 filas completas, combinado con Hash Join contra drivers activos.

Efecto de los índices de optimización

Este es el único caso donde se ve explícitamente un Index Scan reemplazando al Seq Scan (usando el índice preexistente idx_trips_departure, no uno de los 5 nuevos). Aun así la ganancia es marginal (598.7ms → 589.3ms, ~1.6%) porque la query sigue haciendo un doble join contra deliveries (400k filas) que domina el costo total.

Antes de índices	Después de índices	Variación
598,711 ms	589,268 ms	▼ 1.6%

Veredicto: MEJORA LEVE

Query 7 — Rutas con mayor consumo de combustible por km

Problema de negocio: *Identificar rutas ineficientes para optimización.*

```
EXPLAIN ANALYZE
SELECT
    r.route_id, r.route_code,
    r.origin_city || ' → ' || r.destination_city AS ruta,
    r.distance_km,
    COUNT(t.trip_id) AS viajes_realizados,
    ROUND(AVG(t.fuel_consumed_liters), 2) AS consumo_promedio_litros,
    ROUND(AVG(t.fuel_consumed_liters) / NULLIF(r.distance_km, 0), 4) AS litros_por_km
FROM routes r
JOIN trips t ON r.route_id = t.route_id
WHERE t.fuel_consumed_liters IS NOT NULL
GROUP BY r.route_id, r.route_code, r.origin_city, r.destination_city, r.distance_km
ORDER BY litros_por_km DESC;
```

Plan de ejecución (antes de índices)

Seq Scan completo sobre trips (100.000 filas) filtrando por fuel_consumed_liters IS NOT NULL, con Hash Join contra las 48 rutas (routes también por Seq Scan).

Efecto de los índices de optimización

Ninguno de los 5 índices cubre fuel_consumed_liters ni la tabla routes de forma útil para este filtro (idx_routes_metrics no incluye fuel_consumed_liters). El optimizador sigue con Seq Scan sobre trips por ser la opción más barata cuando casi todas las filas cumplen el filtro IS NOT NULL. Tiempo estable: 90.6ms → 92.5ms.

Antes de índices	Después de índices	Variación
90,584 ms	92,467 ms	▲ 2.1%

Veredicto: SIN MEJORA

Query 8 — Entregas retrasadas por día de la semana

Problema de negocio: *Identificar patrones de retraso para mejorar planificación.*

```
EXPLAIN ANALYZE
SELECT
    TO_CHAR(d.scheduled_datetime, 'Day') AS dia_semana,
    EXTRACT(DOW FROM d.scheduled_datetime) AS dia_numero,
    COUNT(*) FILTER (WHERE d.delivered_datetime > d.scheduled_datetime) AS entregas_retrasadas,
    ROUND(AVG(
        EXTRACT(EPOCH FROM (d.delivered_datetime - d.scheduled_datetime)) / 3600
    ) FILTER (WHERE d.delivered_datetime > d.scheduled_datetime), 2) AS horas_promedio_retraso
FROM deliveries d
WHERE d.delivered_datetime IS NOT NULL
GROUP BY TO_CHAR(d.scheduled_datetime, 'Day'), EXTRACT(DOW FROM d.scheduled_datetime)
ORDER BY dia_numero;
```

Plan de ejecución (antes de índices)

Seq Scan sobre las 400.000 filas de deliveries, con un Sort externo a disco (external merge, 16.488kB) porque el resultado a ordenar no entra en work_mem. Es la query más costosa del set en tiempo de ejecución (>1 segundo).

Efecto de los índices de optimización

idx_deliveries_scheduled_datetime tiene una cláusula WHERE delivery_status='delivered', pero esta query filtra por delivered_datetime IS NOT NULL, una condición distinta que el índice parcial no cubre — el optimizador no puede usarlo. El cuello de botella real es el Sort Method: external merge (disco), no la falta de índice; el tiempo empeoró levemente (1099.3ms → 1148.6ms), variación atribuible al I/O de disco entre corridas.

Antes de índices	Después de índices	Variación
1.099,299 ms	1.148,556 ms	▲ 4.5%

Veredicto: SIN MEJORA

Query 9 — Costo de mantenimiento por kilómetro recorrido

Problema de negocio: *Evaluar costo-beneficio de cada tipo de vehículo.*

```
EXPLAIN ANALYZE
WITH costos AS (
    SELECT vehicle_id, SUM(cost) AS costo_total
    FROM maintenance
    GROUP BY vehicle_id
),
kilometros AS (
    SELECT t.vehicle_id, SUM(r.distance_km) AS km_total
    FROM trips t
    JOIN routes r ON t.route_id = r.route_id
    GROUP BY t.vehicle_id
)
SELECT
    v.vehicle_id, v.license_plate, v.vehicle_type,
    c.costo_total, k.km_total,
    ROUND(c.costo_total / NULLIF(k.km_total, 0), 2) AS costo_por_km
FROM vehicles v
JOIN costos c ON v.vehicle_id = c.vehicle_id
JOIN kilometros k ON v.vehicle_id = k.vehicle_id
ORDER BY costo_por_km DESC
LIMIT 20;
```

Plan de ejecución (antes de índices)

Dos CTEs materializadas: costos (agregando maintenance, ~5000 filas) y kilometros (agregando trips + routes, 100.000 filas). Ambas se combinan con vehicles mediante Hash Join.

Efecto de los índices de optimización

idx_maintenance_vehicle_cost (vehicle_id, cost) calza exactamente con el GROUP BY de la CTE costos, pero con solo 5000 filas en maintenance el Seq Scan sigue siendo más barato que el acceso por índice. El resultado empeoró levemente (72.1ms → 77.0ms, ~7%), variación normal para una query dominada por el agregado de 100.000 filas en trips, no por maintenance.

Antes de índices	Después de índices	Variación
72,072 ms	77,026 ms	▲ 6.9%

Veredicto: SIN MEJORA

Query 10 — Ranking de conductores por eficiencia (Window Functions)

Problema de negocio: *Identificar top performers para incentivos.*

```
EXPLAIN ANALYZE
WITH metricas_conductor AS (
    SELECT
        dr.driver_id, dr.employee_code,
        dr.first_name || ' ' || dr.last_name AS conductor,
        COUNT(DISTINCT t.trip_id) AS viajes_completados,
        COUNT(del.delivery_id) AS entregas_totales,
        ROUND(AVG(EXTRACT(EPOCH FROM (t.arrival_datetime - t.departure_datetime)) / 3600), 2) AS
horas_promedio_viaje,
        ROUND(SUM(t.fuel_consumed_liters) / NULLIF(SUM(r.distance_km), 0), 4) AS litros_por_km,
        COUNT(del.delivery_id) FILTER (WHERE del.delivered_datetime <= del.scheduled_datetime) AS
entregas_a_tiempo,
        ROUND(
            100.0 * COUNT(del.delivery_id) FILTER (WHERE del.delivered_datetime <=
del.scheduled_datetime)
            / NULLIF(COUNT(del.delivery_id) FILTER (WHERE del.delivered_datetime IS NOT NULL), 0),
2
        ) AS pct_puntualidad
    FROM drivers dr
    JOIN trips t ON dr.driver_id = t.driver_id
    JOIN routes r ON t.route_id = r.route_id
    LEFT JOIN deliveries del ON del.trip_id = t.trip_id
    WHERE dr.status = 'active' AND t.status = 'completed'
    GROUP BY dr.driver_id, dr.employee_code, dr.first_name, dr.last_name
)
SELECT
    driver_id, employee_code, conductor, viajes_completados, entregas_totales,
    horas_promedio_viaje, litros_por_km, pct_puntualidad,
    RANK() OVER (ORDER BY entregas_totales DESC) AS ranking_entregas,
    RANK() OVER (ORDER BY litros_por_km ASC) AS ranking_eficiencia_combustible,
    RANK() OVER (ORDER BY pct_puntualidad DESC) AS ranking_puntualidad
FROM metricas_conductor
ORDER BY ranking_entregas
LIMIT 30;
```

Plan de ejecución (antes de índices)

La CTE hace triple join (drivers + trips + routes) más un LEFT JOIN a deliveries (400.000 filas), todo por Seq Scan/Hash Join. Es la query con mayor tiempo absoluto del manual (>3.4 segundos), por el volumen combinado de las tablas involucradas.

Efecto de los índices de optimización

idx_drivers_status_license filtra los 373 conductores activos rápido, pero el costo real está en el LEFT JOIN contra deliveries completo — ningún índice de los 5 creados cubre esa combinación. El tiempo empeoró de 3420.0ms a 3643.4ms (~6.5%), la variación absoluta más grande del set, coherente con tratarse de la query más pesada y por lo tanto más sensible al estado de la caché de buffers entre corridas.

Antes de índices	Después de índices	Variación
3.419,999 ms	3.643,371 ms	▲ 6.5%

Veredicto: SIN MEJORA

Query 11 — Tendencia de viajes con LAG y LEAD

Problema de negocio: *Proyectar necesidades futuras basadas en tendencias.*

```
EXPLAIN ANALYZE
WITH viajes_por_mes AS (
    SELECT
        DATE_TRUNC('month', departure_datetime) AS mes,
        COUNT(*) AS total_viajes
    FROM trips
    WHERE status = 'completed'
    GROUP BY DATE_TRUNC('month', departure_datetime)
),
con_variacion AS (
    SELECT mes, total_viajes,
        total_viajes - LAG(total_viajes) OVER (ORDER BY mes) AS variacion
    FROM viajes_por_mes
)
SELECT mes, total_viajes, variacion,
    CASE
        WHEN variacion IS NULL THEN 'Sin dato previo'
        WHEN variacion > 0 THEN 'Crecimiento'
        WHEN variacion < 0 THEN 'Caída'
        ELSE 'Estable'
    END AS tendencia
FROM con_variacion
ORDER BY mes;
```

Plan de ejecución (antes de índices)

HashAggregate sobre Seq Scan filtrado de trips (status='completed'), agrupando 100.000 filas en 24 meses distintos, seguido de un Sort liviano para aplicar LAG/LEAD.

Efecto de los índices de optimización

El filtro es por status, no por fecha, así que idx_trips_composite_joins (que empieza por vehicle_id) no ayuda a este acceso. Postgres sigue con Seq Scan sobre trips. Cambio mínimo (59.7ms → 61.1ms, ~2%), dentro del margen de ruido esperado.

Antes de índices	Después de índices	Variación
59,679 ms	61,070 ms	▲ 2.3%

Veredicto: SIN MEJORA

Query 12 — Pivot de entregas por hora y día de la semana

Problema de negocio: *Optimizar horarios de operación y personal.*

```
EXPLAIN ANALYZE
SELECT
    EXTRACT(HOUR FROM scheduled_datetime) AS hora_del_dia,
    COUNT(*) FILTER (WHERE EXTRACT(DOW FROM scheduled_datetime) = 0) AS domingo,
    COUNT(*) FILTER (WHERE EXTRACT(DOW FROM scheduled_datetime) = 1) AS lunes,
    COUNT(*) FILTER (WHERE EXTRACT(DOW FROM scheduled_datetime) = 2) AS martes,
    COUNT(*) FILTER (WHERE EXTRACT(DOW FROM scheduled_datetime) = 3) AS miercoles,
    COUNT(*) FILTER (WHERE EXTRACT(DOW FROM scheduled_datetime) = 4) AS jueves,
    COUNT(*) FILTER (WHERE EXTRACT(DOW FROM scheduled_datetime) = 5) AS viernes,
    COUNT(*) FILTER (WHERE EXTRACT(DOW FROM scheduled_datetime) = 6) AS sabado,
    COUNT(*) AS total_hora
FROM deliveries
GROUP BY EXTRACT(HOUR FROM scheduled_datetime)
ORDER BY hora_del_dia;
```

Plan de ejecución (antes de índices)

GroupAggregate sobre las 400.000 filas de deliveries, con un Sort externo a disco (10.184kB) necesario porque el resultado intermedio no entra en work_mem antes de agrupar.

Efecto de los índices de optimización

Esta query no tiene cláusula WHERE — necesita recorrer y agrupar el 100% de deliveries por EXTRACT(HOUR...), una expresión que ningún índice estándar de los creados cubre (haría falta un índice funcional específico sobre esa expresión). El tiempo empeoró levemente (1384.9ms → 1424.1ms, ~2.8%), otra vez por I/O de disco en el sort externo, no por los índices.

Antes de índices	Después de índices	Variación
1.384,871 ms	1.424,086 ms	▲ 2.8%

Veredicto: SIN MEJORA

4. Resumen comparativo de performance

El objetivo del ejercicio de optimización era lograr una mejora agregada de al menos 20% en el tiempo de ejecución de las 12 queries. El resultado real medido fue el siguiente: el tiempo total sumado de las 12 queries pasó de 7.008,694 ms a 7.300,088 ms, es decir, un **empeoramiento de 4.2%**, no la mejora esperada.

#	Query	Antes (ms)	Después (ms)	Variación	Veredicto
1	Conteo de vehículos por tipo	0.181	0.169	▼ 6.6%	SIN CAMBIO (esperado)
2	Conductores con licencia próxima a vencer (60 días)	0.267	0.239	▼ 10.5%	SIN CAMBIO (esperado)
3	Total de viajes por estado	41.725	39.512	▼ 5.3%	SIN CAMBIO (esperado)
4	Entregas por ciudad destino (últimos 2 meses)	158.797	136.430	▼ 14.1%	MEJORA
5	Conductores activos con viajes completados	82.509	87.894	▲ 6.5%	SIN MEJORA
6	Promedio de entregas por conductor (últimos 6 meses)	598.711	589.268	▼ 1.6%	MEJORA LEVE
7	Rutas con mayor consumo de combustible por km	90.584	92.467	▲ 2.1%	SIN MEJORA
8	Entregas retrasadas por día de la semana	1.099.299	1.148.556	▲ 4.5%	SIN MEJORA
9	Costo de mantenimiento por kilómetro recorrido	72.072	77.026	▲ 6.9%	SIN MEJORA
10	Ranking de conductores por eficiencia (Window Functions)	3.419.999	3.643.371	▲ 6.5%	SIN MEJORA
11	Tendencia de viajes con LAG y LEAD	59.679	61.070	▲ 2.3%	SIN MEJORA
12	Pivot de entregas por hora y día de la semana	1.384.871	1.424.086	▲ 2.8%	SIN MEJORA

De las 12 queries: **2** mostraron una mejora medible en tiempo de ejecución, **3** no mostraron cambio esperado porque el optimizador correctamente descarta los índices sobre tablas pequeñas (vehicles, drivers) o queries sin filtro selectivo, y **7** empeoraron levemente su tiempo de ejecución.

5. Conclusiones y recomendaciones

Por qué los índices no generaron la mejora esperada

• **Desajuste entre índices y filtros reales:** varios índices parciales (WHERE delivery_status='delivered', WHERE status='active') no coinciden exactamente con las condiciones usadas en cada query (por ejemplo, la Query 8 filtra por delivered_datetime IS NOT NULL, no por delivery_status), por lo que el optimizador no puede aprovecharlos.

• **Tablas pequeñas favorecen Seq Scan:** vehicles (200 filas) y drivers (400 filas) son tan chicas que Postgres descarta cualquier índice por costo — leer la tabla completa en una sola pasada es más barato que el overhead de acceso al índice más el heap fetch. Esto es un comportamiento correcto del optimizador, no un fallo de los índices.

• **El cuello de botella real está en operaciones de disco, no en búsqueda:** las queries 8 y 12, las más lentas del set, muestran `Sort Method: external merge` escribiendo a disco porque el resultado intermedio no entra en work_mem. Ningún índice resuelve ese problema: la solución pasa por aumentar work_mem o rediseñar la query para reducir el volumen a ordenar.

• **Variabilidad entre corridas:** las mediciones se tomaron con una sola ejecución de EXPLAIN ANALYZE por escenario. Factores como el estado de la caché de buffers de PostgreSQL, procesos concurrentes del sistema operativo y el propio overhead de mantenimiento de 5 índices adicionales en cada escritura pueden explicar variaciones de 2-7% sin que reflejen una tendencia real.

Recomendaciones para una siguiente iteración

• Repetir cada EXPLAIN ANALYZE 3-5 veces y promediar, para separar la mejora real del ruido de medición — especialmente en las queries que superan 1 segundo de ejecución.

• Validar el uso efectivo de cada índice con `pg_stat_user_indexes` (columna idx_scan) antes de asumir que fueron aprovechados.

• Ajustar las cláusulas WHERE de los índices parciales para que coincidan literalmente con los filtros usados en las queries (por ejemplo, agregar un índice sobre delivered_datetime en vez de delivery_status para la Query 8).

• Evaluar un aumento de work_mem para la sesión de reporting, dado que las queries 8 y 12 dependen de sorts que hoy se resuelven en disco.

• Considerar un índice funcional sobre EXTRACT(HOUR FROM scheduled_datetime) si la Query 12 (pivot por hora) se va a ejecutar con frecuencia en producción.